INTERNET OF THINGS / LAPTOP LAB 1

Transport Layer

TCP, UDP, message framing, and application reliability

Estimated time: 120 minutes **Environment:** one Ubuntu 24.04 LTS VM or native Ubuntu.

Hardware: laptop only; no microcontroller, radio, or cloud account.

Learning outcomes

After this lab, you should be able to (1) identify transport ports and sockets, (2) distinguish TCP's byte stream from UDP datagrams, (3) implement a small acknowledgment/retry mechanism, and (4) interpret delivery, timeout, duplicate, and latency measurements without confusing transport and application behavior.

1 Concepts before commands

A **socket** is an operating-system interface used by an application to send or receive data. An IP address identifies a network interface; a **port** identifies a transport endpoint. Our virtual temperature sensor sends JSON records to a collector on port 5005. The `127.0.0.1` address is **loopback**: both processes run on the same machine.

Feature	TCP	UDP
Data model	Ordered byte stream; application defines records.	Separate datagrams preserve message boundaries.
Loss handling	Retransmission and ordering while the connection remains usable.	No built-in delivery acknowledgment or retransmission.
IoT trade-off	Useful when complete, ordered transfer matters.	Useful when the application can handle loss or prioritize freshness.

Neither protocol itself confirms that a business operation was completed. Here, `ACK` means an *application* acknowledgment after the server records a message in memory. It is separate from a TCP ACK.

Working environment and files

Use a single Ubuntu 24.04 LTS virtual machine (VM) on Windows or macOS, or run directly on Ubuntu. Allocate approximately 2 virtual CPUs, 4 GB RAM, and 25 GB disk space for a desktop VM. Select an Ubuntu image matching your laptop's CPU architecture. NAT networking is sufficient for installing packages; the lab traffic stays within the laptop. Keep all terminals inside the same VM.

Extract the companion ZIP, then open the matching `code/01_transport` folder in a terminal. Alternatively, create an empty working folder and copy each complete program from the appendix into the indicated filename. For example, `nano filename.py` opens an editor; save with Ctrl+O, Enter, then Ctrl+X. Commands beginning with `sudo` ask for your Ubuntu password. Internet access is needed for installation, not for the experiments. Save results before cleanup. This tutorial can be completed independently of the other three.

```
sudo apt update
sudo apt install -y python3 iproute2 tcpdump
python3 --version
```

Copy or locate `transport_lab.py` and `framing_demo.py`. Open terminals A (server), B (client), and C (optional packet capture) in the same folder. Stop a foreground server with Ctrl+C before changing its options. Restart it for each measured run so that its counters start at zero.

2 Experiment 1: two TCP writes are not two records (15 minutes)

Start the receiver in A, then the sender in B:

```
# Terminal A
python3 framing_demo.py server
# Terminal B
python3 framing_demo.py client
```

The sender makes two `sendall()` calls. The receiver intentionally asks for at most four bytes per `recv(4)` call. Its output contains several chunks, followed by the two reconstructed records. Exact chunk boundaries can vary. The newline byte is our record delimiter; TCP does not supply that delimiter.

Checkpoint. The final list contains `temperature=25` and `humidity=60`. Change the receive buffer to 3 and repeat.

Record and explain. Explain why `recv(4096)` cannot safely be interpreted as "one complete sensor reading." Suggest length-prefix framing for binary data.

3 Experiment 2: baseline TCP and UDP telemetry (20 minutes)

Run UDP first:

```
# Terminal A
python3 transport_lab.py server --protocol udp
# Terminal B
python3 transport_lab.py client --protocol udp --out udp_baseline.csv
```

Restart A with TCP and repeat in B:

```
# Terminal A
python3 transport_lab.py server --protocol tcp
# Terminal B
python3 transport_lab.py client --protocol tcp --timeout 5 --out tcp_baseline.csv
```

Both runs should report 20 acknowledged records on an otherwise healthy local machine. The server logs `STORE` once per unique `(run, seq)` pair. The random run identifier keeps records from different clients distinct.

The client sends one reading, waits for its matching application ACK, then continues. This is a **stop-and-wait workload**, not a bulk throughput benchmark. `elapsed_ms` covers all attempts and waiting for one record; it excludes the interval before the next record. It is application round-trip completion time, not one-way network latency.

4 Experiment 3: missing data and bounded retries (25 minutes)

Restart A with the following fault injector, then run B:

```
# Terminal A
python3 transport_lab.py server --protocol udp --drop-every 4 --fault data
# Terminal B
python3 transport_lab.py client --protocol udp --retries 0 --out udp_drop.csv
```

The collector deliberately ignores the *first arrival* of sequence numbers 4, 8, 12, 16, and 20. This is deterministic **application-level fault injection**, not a measurement of radio loss and not a fault in UDP. The socket has already received those datagrams when the program discards them.

Restart the same server and enable three additional attempts per record:

```
python3 transport_lab.py client --protocol udp --retries 3 --out udp_retry.csv
```

Checkpoint. Without other failures, no retry gives 15/20 acknowledged and 15 stored records. With retries, expect 20/20 acknowledged, 20 stored records, and 25 application sends. The five affected records each wait about one 250 ms timeout before their successful retry.

Record and explain. Compare median and maximum completion times. Change `--timeout` to 0.10 and 0.50 seconds. Why can a shorter timeout create unnecessary retries on a real network? Why are bounded retries preferable to retrying forever?

5 Experiment 4: losing an ACK does not undo a reading (20 minutes)

Restart A with `--fault ack`; first test without retries:

```
# Terminal A
python3 transport_lab.py server --protocol udp --drop-every 4 --fault ack
# Terminal B
python3 transport_lab.py client --protocol udp --retries 0 --out ack_drop.csv
```

The collector now stores the record but suppresses its first ACK. Expect 20 stored records but only 15 acknowledged records. A client timeout means the client does not know whether processing succeeded.

Restart A again, repeat with `--retries 3 --out ack_retry.csv`, and inspect `DUPLICATE` lines. The collector acknowledges duplicate arrivals without storing a second logical record. Expect 20 unique records, 25 sends, and five duplicate arrivals. This in-memory deduplication disappears on server restart. It is not durable exactly-once execution.

6 Experiment 5: inspect the transport headers (15 minutes)

Restart a clean TCP server. In C start a capture; run the TCP client in B; then stop the capture with Ctrl+C. Repeat for UDP using a second capture file.

```
# Terminal C: use a different output filename for each run
sudo tcpdump -i lo -nn -s 0 -U -w /tmp/iot_tcp.pcap 'port 5005'
# After Ctrl+C, read the capture without a graphical tool:
tcpdump -nn -r /tmp/iot_tcp.pcap
```

For UDP use `/tmp/iot_udp.pcap`. If the capture belongs to a different account, read it with `sudo tcpdump -nn -r ...`. Look for the TCP SYN/SYN-ACK/ACK handshake, source and destination ports, and connection close. UDP has no connection handshake. Optional Wireshark display filters are `tcp.port == 5005` and `udp.port == 5005`. Capture on `lo`, not the VM's Internet adapter. A loopback capture does not show over-the-air frames. Do not use packet counts or checksum-offload artifacts as direct evidence of radio overhead or corruption.

7 Results and questions (25 minutes)

Fill in this table from your files and server logs. Expected counts apply only to the deterministic local experiment; timings are measured, not prescribed.

Run	ACKed	Unique	Sends	Max ms
UDP baseline				
TCP baseline				
UDP data loss, no retry				
UDP data loss, retries				
UDP ACK loss, no retry				
UDP ACK loss, retries				

For each run calculate

$$\text{ACK success ratio} = \frac{\text{acknowledged records}}{20}, \qquad \text{retry overhead} = \frac{\text{application sends} - 20}{20}.$$

This overhead excludes transport headers and TCP's internal retransmissions.

1. Which run proves that "no ACK" does not mean "not stored"?
2. If a TCP application discards a received command, will TCP resend that command automatically? Explain the boundary between transport and application.
3. Why do sequence numbers alone become ambiguous after a device restart?
4. For periodic temperature readings and a valve-opening command, specify different reliability and freshness requirements. Do not choose solely by which protocol has the lower local timing.
5. What must change for durable deduplication and multiple concurrent clients?

Optional extension: actual IP packet loss

After completing Tutorial 2, keep its three namespaces active. Start the same server and client across them; the absolute path ensures Python finds the file. These commands are run from the folder containing `transport_lab.py`.

```
# Terminal A
sudo ip netns exec iot-cloud python3 "$PWD/transport_lab.py" server \
  --protocol udp --host 10.10.2.2
# Terminal B: impair only the sensor's outgoing virtual link
sudo ip netns exec iot-sensor tc qdisc replace dev s0 root netem \
  delay 40ms loss 10%
sudo ip netns exec iot-sensor python3 "$PWD/transport_lab.py" client \
  --protocol udp --host 10.10.2.2 --count 100 --out udp_netem.csv
```

Repeat with retries, then change both programs to TCP and use `--timeout 30`. Restart the server between runs. No `--drop-every` option is needed: the kernel now drops packets before application delivery. This one-direction impairment is not a complete wireless channel model. Random results vary; TCP may recover at increased delay or fail after a sufficiently long outage. Inspect actual retransmissions with a namespace packet capture. Remove the impairment before continuing:

```
sudo ip netns exec iot-sensor tc qdisc del dev s0 root
```

Submission, troubleshooting, and cleanup

Submit a short report with the completed table, two annotated capture excerpts, answers, your Python code, and the six CSV files. Clearly label simulated faults. No particular laptop-dependent latency value is required.

Connection refused: start the matching server first. **Address already in use:** stop the old process or choose another matching port. **Unexpected counts:** restart the server before every run and check `--fault`, `--drop-every`, and `--retries`. **Capture denied:** use `sudo` inside your VM. Stop all foreground programs with Ctrl+C; the core lab does not change your network configuration.

References

1. IETF, RFC 9293, *Transmission Control Protocol*, 2022. https://www.rfc-editor.org/rfc/rfc9293.html
2. IETF, RFC 768, *User Datagram Protocol*, 1980. https://datatracker.ietf.org/doc/html/rfc768
3. Python documentation, *socket: Low-level networking interface*. https://docs.python.org/3/library/socket.html
4. iproute2 manual, *tc-netem*. https://man7.org/linux/man-pages/man8/tc-netem.8.html

A Complete source: transport_lab.py

The collector accepts this lab's trusted, small JSON messages and handles one TCP connection at a time. It is intentionally not a production service.

```
#!/usr/bin/env python3
"""Small TCP/UDP telemetry experiment. Python 3 standard library only."""
import argparse
import csv
import json
import socket
import time
import uuid


def encode(obj):
    return (json.dumps(obj) + "\n").encode()


def serve(a):
    seen, attempted = set(), set()

    def process(raw):
        msg = json.loads(raw)
        key = (msg["run"], msg["seq"])
        first = key not in attempted
        attempted.add(key)
        fault = (a.protocol == "udp" and a.drop_every > 0
                 and msg["seq"] % a.drop_every == 0 and first)
        if fault and a.fault == "data":
            print("SIMULATED_DATA_DROP", key, flush=True)
            return None
        duplicate = key in seen
        seen.add(key)
        print("DUPLICATE" if duplicate else "STORE", key,
              "total_unique=", len(seen), flush=True)
        if fault and a.fault == "ack":
            print("SIMULATED_ACK_DROP", key, flush=True)
            return None
        return encode({"run": msg["run"], "seq": msg["seq"]})

    kind = socket.SOCK_DGRAM if a.protocol == "udp" else socket.SOCK_STREAM
    with socket.socket(socket.AF_INET, kind) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind((a.host, a.port))
        print("READY", a.protocol, a.host, a.port, flush=True)
        if a.protocol == "udp":
            while True:
                raw, peer = s.recvfrom(4096)
                reply = process(raw)
                if reply is not None:
```

```
                    s.sendto(reply, peer)
        else:
            s.listen(1)
            while True:
                conn, _ = s.accept()
                with conn, conn.makefile("rb") as stream:
                    for raw in stream:  # Newline framing, not recv boundaries.
                        conn.sendall(process(raw))


def send(a):
    run = uuid.uuid4().hex[:8]
    kind = socket.SOCK_DGRAM if a.protocol == "udp" else socket.SOCK_STREAM
    rows = []
    with socket.socket(socket.AF_INET, kind) as s:
        s.settimeout(a.timeout)
        s.connect((a.host, a.port))
        stream = s.makefile("rb") if a.protocol == "tcp" else None
        try:
            for seq in range(1, a.count + 1):
                raw = encode({"run": run, "seq": seq,
                              "temperature_c": round(24 + seq / 10, 1)})
                start = time.perf_counter()
                acknowledged = False
                limit = a.retries + 1 if a.protocol == "udp" else 1
                for attempt in range(1, limit + 1):
                    if a.protocol == "tcp":
                        s.sendall(raw)
                        reply = stream.readline()
                        if not reply:
                            raise ConnectionError("Server closed the connection")
                        ack = json.loads(reply)
                        acknowledged = (ack["run"], ack["seq"]) == (run, seq)
                    else:
                        s.send(raw)
                        deadline = time.perf_counter() + a.timeout
                        while time.perf_counter() < deadline:
                            s.settimeout(max(0.001, deadline - time.perf_counter()))
                            try:
                                ack = json.loads(s.recv(4096))
                            except socket.timeout:
                                break
                            if (ack["run"], ack["seq"]) == (run, seq):
                                acknowledged = True
                                break
                    if acknowledged:
                        break
                elapsed = 1000 * (time.perf_counter() - start)
                rows.append([seq, attempt, int(acknowledged), round(elapsed, 3)])
                time.sleep(a.interval)
        finally:
            if stream:
                stream.close()
    with open(a.out, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["seq", "attempts", "acknowledged", "elapsed_ms"])
        writer.writerows(rows)
    ok = sum(row[2] for row in rows)
    print(f"run={run} acknowledged={ok}/{a.count}")
    print("application_sends=", sum(row[1] for row in rows))
    print("CSV:", a.out)


def main():
    p = argparse.ArgumentParser()
    p.add_argument("role", choices=["server", "client"])
    p.add_argument("--protocol", choices=["tcp", "udp"], default="udp")
    p.add_argument("--host", default="127.0.0.1")
    p.add_argument("--port", type=int, default=5005)
    p.add_argument("--count", type=int, default=20)
    p.add_argument("--retries", type=int, default=0)
    p.add_argument("--timeout", type=float, default=0.25)
    p.add_argument("--interval", type=float, default=0.02)
```

```
    p.add_argument("--drop-every", type=int, default=0)
    p.add_argument("--fault", choices=["data", "ack"], default="data")
    p.add_argument("--out", default="results.csv")
    a = p.parse_args()
    if a.count < 1 or a.retries < 0 or a.timeout <= 0 or a.interval < 0:
        p.error("Use positive count/timeout and nonnegative retries/interval")
    try:
        serve(a) if a.role == "server" else send(a)
    except KeyboardInterrupt:
        print("Stopped")
    except (OSError, ConnectionError) as error:
        p.exit(1, f"Communication failed: {error}\n")


if __name__ == "__main__":
    main()
```

B Complete source: framing_demo.py

```
#!/usr/bin/env python3
"""TCP demonstration: two sendall calls are not two receive records."""
import argparse
import socket

p = argparse.ArgumentParser()
p.add_argument("role", choices=["server", "client"])
a = p.parse_args()
with socket.socket() as s:
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    if a.role == "server":
        s.bind(("127.0.0.1", 5006))
        s.listen(1)
        print("READY on port 5006", flush=True)
        conn, _ = s.accept()
        with conn:
            chunks = []
            while True:
                part = conn.recv(4)
                if not part:
                    break
                print("recv:", repr(part))
                chunks.append(part)
            print("records:", b"".join(chunks).splitlines())
    else:
        s.connect(("127.0.0.1", 5006))
        s.sendall(b"temperature=25\n")
        s.sendall(b"humidity=60\n")
        s.shutdown(socket.SHUT_WR)
```